

Interview Questions — SE - Introduction to Programming

Total Questions: 47

1. What is a program, and what is a programming language?

Hint: One is a set of instructions, the other is the tool to write them.

2. Explain the difference between a compiler and an interpreter.

Hint: Think about how and when each translates source code.

3. Why are algorithms and flowcharts important before starting to write code?

Hint: Focus on planning and clarity.

4. Draw a simple flowchart to represent the process of ordering food online.

Hint: Identify the main steps from choosing food to receiving the order.

5. Describe the basic structure of a C program.

Hint: Include the header, main function, and return statement.

6. How do you compile and run a C program using GCC or the VS Code terminal?
-

7. Write a simple C program that prints 'Hello, TOPS Technologies!' to the screen.
-

8. List four basic data types in C and give a short example of each.

Hint: Think of whole numbers, decimals, characters, and larger decimals.

9. What are the rules for naming variables in C?

Hint: Consider allowed characters, starting characters, and reserved words.

10. What is the difference between a constant and a variable in C?
-

11. Write a C code snippet to store and print a student's name, age, and marks.
-

12. What is the difference between arithmetic, logical, and relational operators in C?

Hint: Think about what each operator type is used to compare or calculate.

13. What does operator precedence mean, and why is it important?

Hint: Consider how C chooses the order of operations in an expression.

14. Write a simple calculator program in C that can add and subtract two numbers.

15. Explain the difference between if, else if, and switch-case statements.

16. Give an example of a nested if statement and explain when you might use it.

17. Write a C code snippet that takes user input and prints a motivational message using if-else logic.

18. What is the main difference between entry-controlled and exit-controlled loops?

Hint: Think about when the loop condition is checked.

19. Write a C program using a for loop to print numbers from 10 down to 1.

20. When would you use a do-while loop instead of a while loop?

21. How do nested loops help in creating grid or pattern outputs?

Hint: Consider how one loop handles rows and another handles columns.

22. Write a C program using nested loops to print the following pattern: * ** *** ****

23. What is a function in C, and why do we use them?

Hint: Think about code reuse and organization.

24. Explain the difference between pass-by-value and pass-by-reference in C.

25. Write a C function that takes two numbers as arguments and returns their sum.

26. What is an array, and how is a 1D array different from a 2D array?

27. Write a C program to record the screen time for 7 days in an array and calculate the average.

28. How would you declare and initialize a string in C?

29. What does the function `strlen()` do? Give an example.

30. Write a C code snippet to create a username by taking the first four letters of a user's name.

31. What is a pointer in C, and how is it different from a normal variable?

Hint: Think about memory addresses.

32. Explain pointer arithmetic with an example.

33. Write a C function to swap two numbers using pointers.

34. How do you declare and initialize a structure in C?

Hint: Think about grouping different data types together.

35. Give an example of a nested structure and explain a use case for it.

36. Write a C structure to store a student's name, marks, and grade.

37. What are the basic steps to open, write to, read from, and close a file in C?

38. Explain the difference between 'r', 'w', and 'a' modes when opening a file.

39. Write a C program that writes your daily goals to a file called 'daily_goals.txt' and then reads them back.

40. What is the difference between a syntax error and a runtime error?

Hint: Think about when the error is detected.

41. Why is commenting and indentation important in programming?

42. Suppose your code compiles but the output is not what you expect. How would you start debugging it?

Hint: Think about checking logic, print statements, and input values.

43. Describe how you would design a menu-driven console application for tracking student productivity.

44. How would you combine arrays, loops, and file handling to log daily study hours and generate a weekly progress report?

45. If you were building the 'Student Productivity Tracker' project, how would you ensure data is saved and loaded correctly each time the program runs?

46. How could you use ChatGPT or GitHub Copilot to help you write or debug C programs, and how would you check that the AI's suggestions are correct?

Hint: Think about using AI for code examples and reviewing the output yourself.

47. What are some ways AI tools could help you generate flowcharts or plan algorithms before coding, and why should you still understand the logic yourself?
